Can a clinic afford to run its own ICD-10 model?

Dr. Findus, our startup in Berlin

German GPs code every consultation.

We propose, the physician confirms.

Reliability is the product.

Today it runs on hosted APIs.

Could it run on our own hardware?

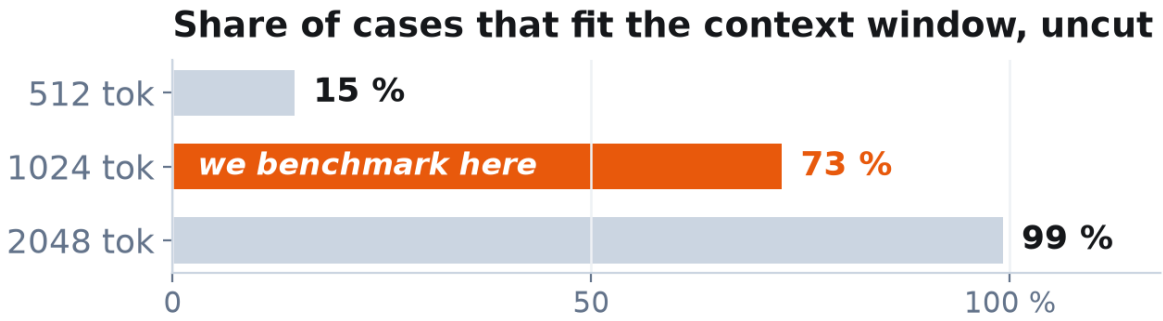
Qwen3-4B + LoRA on CodiEsp: 500 train / 250 dev
Spanish case reports, ICD-10 coded, CC-BY 4.0.

The resource problem

8 GB of weights at 4B before a single activation.

Median case 802 tokens, longest 2,489.

One JAX program, lowered by XLA to CPU, GPU and TPU.



Where does the time go, and what does it take to run this at all?

APPROACH

One program. Three backends. All measured.

Compilation

One JAX program

XLA lowers it to all three

LoRA via qwix, Tunix trainer

Orchestration

3 pinned Docker images

Kubernetes across 2 clusters

Kueue admission, explicit limits

Storage

gcsfuse CSI, implicit-dirs

8 GB checkpoint from a bucket

File cache left as a knob

gs://me344-tpu-labs-west4 → gcsfuse CSI sidecar → /gcs in the pod → 8 GB checkpoint → HBM

Backend	Hardware	Host arch	How the code got there
CPU	32-core x86_64, 31 GiB	x86_64	bare node, podman
GPU	NVIDIA GH200 480 GB	ARM64 Grace	public image + ConfigMap
TPU	v5e 2×4, 8 chips	x86_64	private registry + gcsfuse

The GH200 host is ARM64. That one fact cost five separate blockers.

We did not time the job. We took it apart.

What we sample	Instrument
Chip utilisation	nvidia-smi 1 Hz · psutil on CPU
Peak memory	jax memory_stats() · peak RSS
Step time	median, p10, p90, warmup excluded
Wall clock	six phases, must sum to total

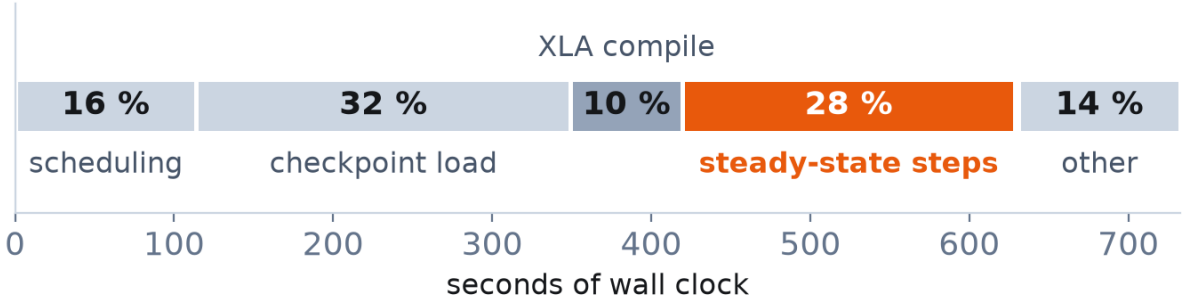
One schema, one JSON per run. Every figure in this deck reads those files and nothing else.

The schema caught a lie

93 µs per step. 11 M tokens/s. Impossible, yet it carried status "ok".

Its own notes admitted it may have timed dispatch, not completion. Discarded and re-run.

Only 28 % of 733 s is arithmetic



Backend	Peak memory, largest fitting run	Next step up
CPU 32-core	66.1 % of 31 GiB (RSS)	4B never ran
GH200	67.8 % of 96 GiB HBM	batch 64 OOM
v5e, 8 chips	65.5 % of 16 GiB/chip	batch 32 OOM

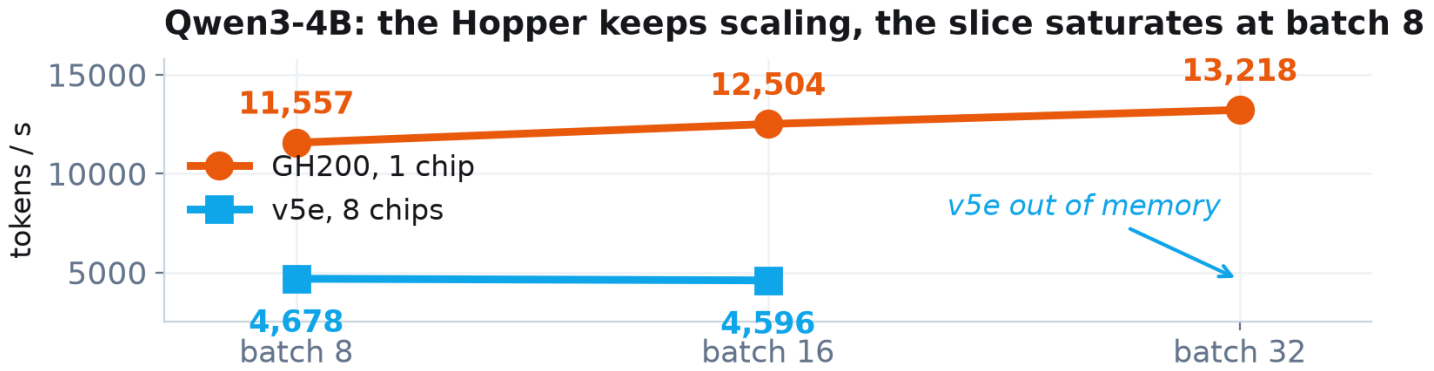
Memory is what ends every sweep. One batch step past these numbers, all three fail.

RESULTS

Under load, one GH200 beats eight TPU chips.

Qwen3-0.6B, batch 1	CPU 32-core	GH200 1 chip	v5e 8 chips
Median step time	18.99 s	0.0798 s	0.0800 s
Speedup vs CPU	1.0×	238×	237×
Mean chip utilisation	44.3 %	1.59 %	no counter exists
Peak memory	66.1 %	5.0 %	7.3 %

Both accelerators are ~238× the CPU and tie with each other to 0.23 %. At batch 1 neither is busy. That tie is an artefact.



2.7x

the throughput of an eight-chip v5e slice, from a single Hopper chip

Where each one runs out of memory

GH200 between batch 32 and 64
v5e between batch 16 and 32

The mitigation, controlled

Checkpoint load 14.6× faster, 31 % off wall clock, and median step time unchanged to the fourth decimal.

CONCLUSION

The chip was never the bottleneck.

28%

of the wall clock is arithmetic

Of 733 s, 209 s computes. The largest single phase is reading the checkpoint: 32 %.

6.7× more parameters, and the CPU stopped

Not 6.7× slower. XLA never finished compiling, and without remat the kernel killed it at 31.5 GB.

1 of 36 dependencies was TPU-bound

The GH200 sat idle behind an ARM64 host, a QEMU segfault and a registry 403. One pin is the whole port.

Do not scale out. Amortise.

Persistent compile cache · warm workers, not a pod per job
file cache on anything that reads a checkpoint

$0.56 against $5.72

per 1,000 steps at 4B, batch 8: one GH200 against eight v5e chips, 10× less

TPU rate billed; the GH200 rate is a market proxy

13 measured runs · github.com/RN0L/qwen3-icd10-scaling